```
  private final PolarProperties polarProperties;          ❶

  public HomeController(PolarProperties polarProperties) {
    this.polarProperties = polarProperties;
  }

  @GetMapping("/")
  public String getGreeting() {
    return polarProperties.getGreeting();                 ❷
  }
}
```

❶ Bean to access the custom properties injected via constructor autowiring

❷ Uses the welcome message from the configuration data bean

You can now build and run the application to verify that it works as intended (`./gradlew bootRun`). Then open a Terminal window, and send a `GET` request to the root endpoint exposed by Catalog Service. The result should be the message you configured in application.yml for the `polar.greeting` property:

```
$ http :9001/
Welcome to the local book catalog!
```

NOTE Property files packaged with your application code are useful for defining sensible default values for your configuration data. They can also act as specifications for what configuration properties your application supports.

The following section will cover the other key aspect modeled by the Spring `Environment` abstraction: profiles, and how to use them for cloud native applications. Before moving on, you can stop the application with Ctrl-C.

4.1.2 Profiles: Feature flags and configuration groups